Ui

Mammoth MVP — Wireframe Screens

I'll number these so they map cleanly to routes/components later.

SCREEN 1 — Landing / Discovery (`/`)

Goal:
Immediate casino energy + credibility. Get users clicking into projects.

```
-------------------------------------------------
| Mammoth logo        Launch Project   Connect |
-------------------------------------------------

[ Tabs / Sort ]
Newest | Fastest Growing | Most Raised | Ending Soon

-------------------------------------------------
| Token Card 1                                  |
| ---------------------------------------- |
| [TOKEN NAME] ($TICKER)       [OPEN]           |
| Current Price: 0.0023 SOL                     |
| Progress: ████████▒▒ 78%                      |
| Mini Chart (sparkline)                        |
|                    [ View ]                   |
-------------------------------------------------

(repeat cards)
```

Notes

- No wallet required
- Cards feel very pump.fun familiar
- Status badge is critical (OPEN is the hook)

SCREEN 2 — Project Page (Open Cycle) (`/token/:mint`)

Goal:
All important info visible immediately. Buying is obvious.

2A. Header + Chart (Above the Fold)

```
-----------------------------------------------
| $TICKER — Token Name                        |
| Creator: AbC...123                          |
| Status: 🟢 Cycle Open                       |
-----------------------------------------------

[ LARGE PRICE CHART ]
```

2B. Cycle Panel (Right or Below Chart)

```
-----------------------------------------------
| CURRENT CYCLE                               |
| ------------------------------------------- |
| Curve: Step (Recommended)                   |
| Allocation Remaining: 42,000 / 100,000      |
| Current Price: 0.0023 SOL                   |
| Next Price Jump In: 3,200 tokens 🔥          |
|                                             |
| Your Rights Remaining: 5,400 tokens         |
|                                             |
| [ BUY TOKENS ]                              |
-----------------------------------------------
```

Notes

- “Next Price Jump” is the gamification heartbeat
- Rights shown only if wallet connected
- No math exposed

2C. Buy Modal

```
---------------------------------------
| Buy $TICKER                         |
---------------------------------------
| Amount (tokens): [ 5,000        ]   |
```

```
|                                    |
| You Pay: 11.8 SOL                  |
| Mammoth Fee (2%): 0.24 SOL         |
| Price Impact: +1.3%                |
| Rights Used: 5,000 / 5,400         |
|                                    |
| Slippage: [ 1% ▼ ]                 |
|                                    |
| [ Confirm Purchase ]               |
--------------------------------------
```

Clear, fast, familiar.

2D. About / Tokenomics Tab

```
------------------------------------------------
| ABOUT | TOKENOMICS | CYCLES | TREASURY       |
------------------------------------------------
```

[Project Description - Markdown]

Supply Mode: Fixed
Total Supply: 1,000,000,000
Hard Cap: Yes (Final)

Rights Policy:
- Snapshot at cycle open
- Non-transferable
- Expire after cycle

Treasury Routing:
- 70% Creator
- 20% Reserve
- 10% Sink

SCREEN 3 — Project Page (Closed Cycle)

Goal:
Keep users trading on Mammoth.

| Status: 🔴 Cycle Completed |

[Price Chart]

[BUY] [SELL]

(Trades routed via aggregator)
Fee notice:
“2% fee applies only when trading via Mammoth”

SCREEN 4 — Creator Dashboard (`/creator`)

Goal:
Everything a creator needs, nothing extra.

| Creator Dashboard |

Token: $TICKER

Supply:
- Minted: 420,000,000
- Remaining: 580,000,000

Active Cycle:
- Status: OPEN
- Sold: 78%
- Curve: Step
- [End Cycle]

Past Cycles:

| ID | Allocation | Status | Raised |

| 1 | 100k | Completed | 240 SOL|
| 2 | 80k | Terminated | 32 SOL |

Treasury Balance:

- SOL: 312.4

SCREEN 5 — Launch Project Wizard (`/launch`)

Goal:
Launch in <5 minutes.

Step 1 — Basics

Token Name: [_____]
Ticker: [____]
Description: [markdown textarea]

Step 2 — Supply Mode

(o) Fixed Supply (Recommended)
( ) Elastic Supply (Advanced ⚠)

[Info Box]
Elastic supply requires rights-based issuance.
Hard cap can be set later (irreversible).

Step 3 — Allocation

Public Allocation: [600,000,000]
Treasury Allocation: [400,000,000]

[Live validation ✓]

Step 4 — Review & Deploy

Token: $TICKER
Supply Mode: Fixed
Public: 600M
Treasury: 400M

[Deploy Token]

SCREEN 6 — Create Cycle (`/creator/cycle/new`)

Goal:
Make cycle setup intuitive, not scary.

6A. Allocation

Tokens for this cycle: [100,000]
Remaining Supply: 500,000

6B. Curve Selection

Curve Type:
(*) Step (Recommended 🔥)
( ) Linear
( ) Exp-Lite

[LIVE PRICE PREVIEW CHART]

Step-specific sliders:

Tokens per step: [5,000]
Price increase per step: [0.0002 SOL]

6C. Rights

Rights-Based Issuance:
(✓) Enabled

[Info]
Rights snapshot taken at cycle open.

6D. Treasury Routing

Creator Treasury: 70%
Reserve: 20%
Sink/Burn: 10%

Protocol Fee: 2% (fixed)

6E. Create Cycle

[Create Cycle (DRAFT)]

SCREEN 7 — Open Cycle Confirmation

⚠ Opening this cycle will:
- Lock all parameters
- Take a rights snapshot
- Make the cycle public

[Confirm & Open Cycle]

SCREEN 8 — End Cycle Early (Modal)

⚠ End Cycle Early?

Unminted tokens will remain unavailable
unless reallocated to a future cycle.

[Cancel] [End Cycle]

What You Have Now

You now have:

- A **complete UI map**
- Pump.fun–familiar flow
- Clear gamification points

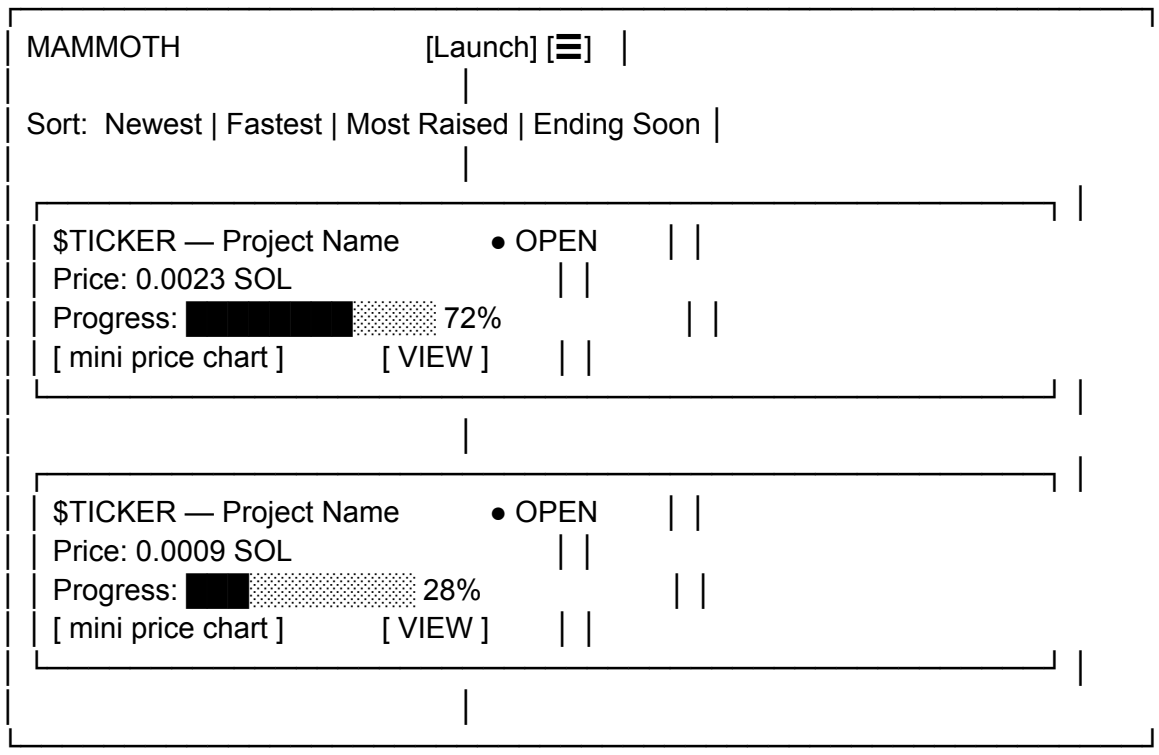
- Minimal cognitive overhead
- No governance clutter

This is **absolutely buildable** with:

- Next.js
- Wallet Adapter
- Jupiter SDK
- Loveable AI for layout + components

Mammoth MVP — Low-Fidelity Mockups

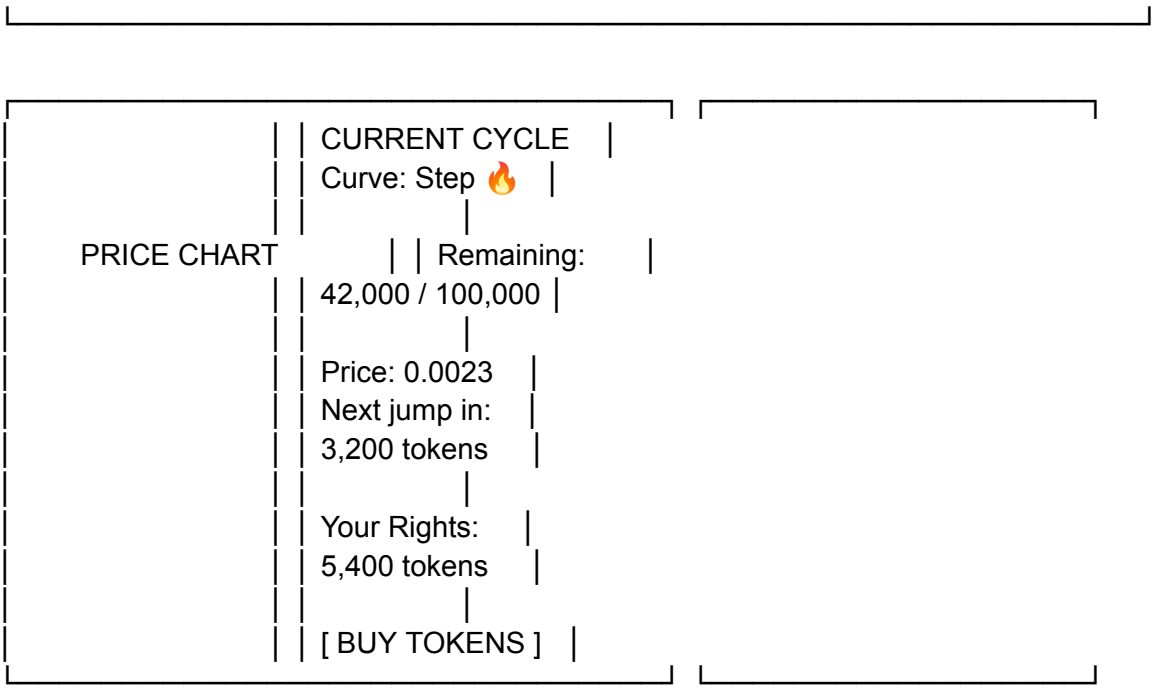
1️⃣ Landing / Discovery



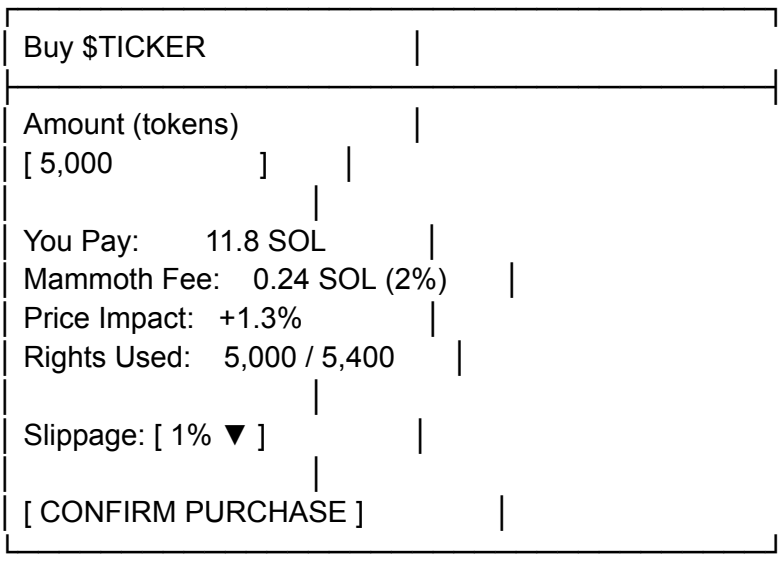
Notes

- No wallet required
- Status dot = dopamine
- Everything clickable

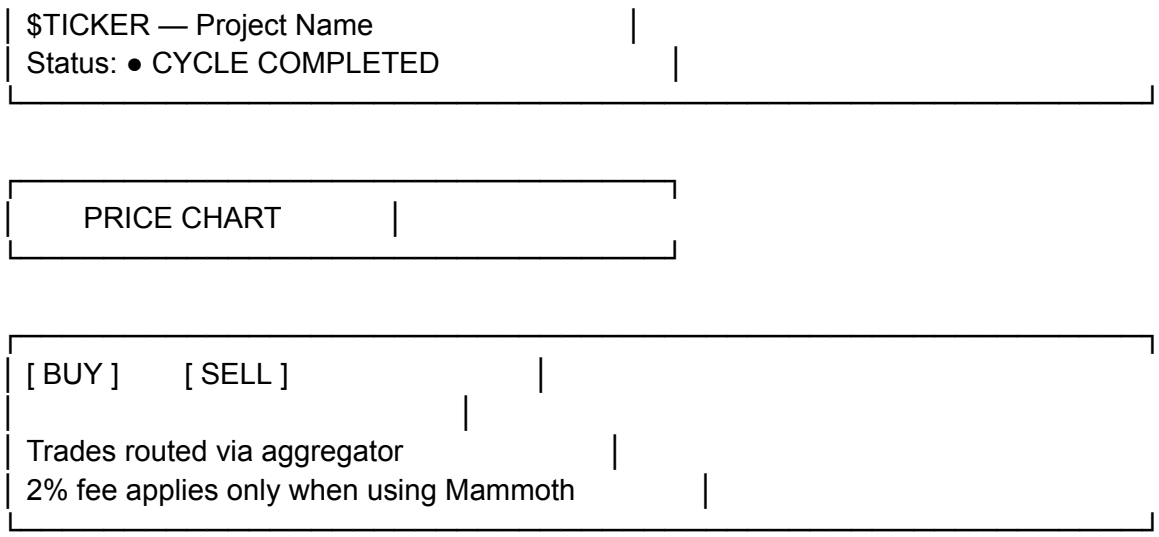
2️⃣ Project Page (Open Cycle)



3️⃣ Buy Modal



4️⃣ Project Page (Closed Cycle / Trading)



---

5 About / Tokenomics Tab

Project Description

[markdown text block]

Supply Mode: Fixed
Total Supply: 1,000,000,000
Hard Cap: Final

Rights:
- Snapshot at cycle open
- Non-transferable
- Expire after cycle

Treasury Routing:
- Creator: 70%
- Reserve: 20%
- Sink: 10%

---

6 Creator Dashboard

```
┌──────────────────────────────────────────────────────────────────────────┐
│ Creator Dashboard                                                        │
└──────────────────────────────────────────────────────────────────────────┘
```

Token: $TICKER

Supply

Minted: 420,000,000
Remaining: 580,000,000

Active Cycle

Status: OPEN
Sold: 78%
Curve: Step
[END CYCLE]

Past Cycles

ID	Allocation	Status	Raised
1	100k	Completed	240 SOL
2	80k	Terminated	32 SOL

Treasury

SOL Balance: 312.4

---

7 Launch Project Wizard

Step 1 — Basics

Token Name: [_____]
Ticker: [____]
Description:
[markdown textarea]

[NEXT]

Step 2 — Supply Mode

(o) Fixed Supply (Recommended)
( ) Elastic Supply (Advanced ⚠)

Elastic supply requires rights-based issuance.
Hard cap can be set later (irreversible).

[NEXT]

Step 3 — Allocation

Public Allocation: [600,000,000]
Treasury Allocation: [400,000,000]

✓ Allocation valid

[DEPLOY TOKEN]

8️⃣ Create Cycle Flow

Allocation

Tokens for this cycle: [100,000]
Remaining supply: 500,000

Curve

Curve Type:
(*) Step (🔥 Recommended)
( ) Linear
( ) Exp-Lite

[Live price preview chart]

Rights

Rights-Based Issuance:
[✓] Enabled
Snapshot taken at cycle open

Treasury Routing

Creator: 70%
Reserve: 20%
Sink: 10%
Protocol: 2% (fixed)

Create

[CREATE CYCLE (DRAFT)]

9️⃣ Open Cycle Confirmation

⚠ Opening this cycle will:
- Lock all parameters
- Take a rights snapshot
- Make the cycle public

[CANCEL] [OPEN CYCLE]

What You Can Do *Right Now*

You can:

- Paste these directly into **Loveable AI**
- Recreate them in **Figma** in ~1 hour
- Hand them to a frontend dev as-is

These mockups already:

- match pump.fun muscle memory
- surface gamification clearly
- align exactly with the Cycle Manager spec
- avoid governance / clutter

Mammoth Visual Style Guide (MVP)

Design Goal

> **“Feels like a casino, reads like a financial instrument.”**

Mammoth should:

- feel familiar to meme traders
- feel credible to repeat builders
- avoid looking like DeFi governance sludge
- avoid looking like NFT art galleries

---

1. Overall Aesthetic

Core Vibe

- **Dark mode first**
- High contrast
- Minimal chrome
- Data-forward
- Sharp edges, not rounded-toy UI

Think:

- pump.fun
- Birdeye
- early Solana dashboards
- NOT Juicebox
- NOT DAOhaus
- NOT NFT marketplaces

---

2. Color System

Base Colors

- **Background:** near-black `#0B0E11` or `#0E1117`
- **Panels:** dark gray `#141821`
- **Borders:** subtle contrast `#1F2633`

Accent Colors (Very Important)

Use **one primary accent** and **one danger color**.

Primary Accent (Mammoth Orange)

- `#FF9F1C`
- Used for:
  - Buy buttons
  - Active states
  - Progress bars
  - “Next price jump” indicators

Danger / Downside

- `#FF4D4D`
- Used for:
  - Sell
  - Cycle termination
  - Warnings

Success / Up

- `#00E676` (sparingly)
- Used for:
  - Completed cycles
  - Price up indicators

⚠️ Rule:
Never use more than 3 colors on one screen.
Most screens should be grayscale + accent.

---

3. Typography

Font Stack

- **Primary:** Inter
- **Fallback:** system-ui

Why:

- Neutral
- Crypto-standard
- Excellent readability for numbers

Hierarchy

- Token name: **bold, large**
- Prices: **medium weight**
- Metadata: **small, muted**
- No playful fonts
- No serif fonts
- No graffiti fonts

This is a **casino for adults**, not a meme coin logo contest.

---

4. Buttons & CTAs

Primary CTA (Buy / Open Cycle)

Background: #FF9F1C
Text: black
Border radius: 4–6px
Font weight: 600

Secondary CTA (Sell / End Cycle)

Background: transparent
Border: 1px solid #FF4D4D
Text: #FF4D4D

Disabled

- Lower opacity
- No animation
- No hover

Hover States

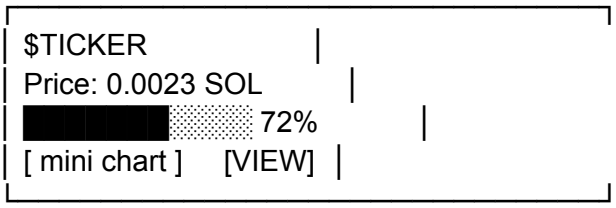
- Slight brightness increase
- No bouncy animations
- No gradients

---

5. Cards & Panels

Token Cards

- Flat rectangles
- Thin border
- Subtle hover glow (accent color at 10–15% opacity)



No shadows.
Depth is created by contrast, not drop-shadow spam.

---

6. Charts

Chart Style

- Dark background
- Thin gridlines
- Accent color for price line
- Red/green volume bars
- Clear cycle boundaries (vertical markers)

Special Gamification Elements

- “Next price jump” indicator (Step curve):
    - Highlighted in accent color

    - Often paired with 🔥 emoji
- Cycle start/end markers

Charts should **sell urgency without noise**.

---

7. Icons & Emoji Usage

Icons

- Use simple outline icons (Lucide / Heroicons)
- No filled cartoon icons

Emoji Rules

Allowed (sparingly):

- 🔥 (step curve, urgency)
- ⚠️ (warnings)
- 🟢 / 🔴 (status)

Never:

- 💎🙌
- 🚀🚀🚀
- 😂🤣
- Meme spam

If emoji feel cringe, remove them — they're optional seasoning, not the meal.

---

8. Copy Tone

Voice

- Short
- Direct
- Neutral
- Slightly irreverent, never jokey

Examples:

- “Next price jump in 2,300 tokens”
- “Rights snapshot taken at cycle open”
- “Unminted tokens remain unavailable”

Avoid:

- hype slogans
- marketing speak
- DAO jargon

---

9. Animations

Allowed

- Progress bar filling
- Price number ticking upward
- Subtle hover glow

Not Allowed

- Page transitions
- Confetti
- Shake animations
- Sound effects

This is **calculated risk**, not mobile gaming.

---

10. Accessibility & Legibility

- Font sizes never below 12px
- High contrast ratios
- Numbers always monospace-aligned
- Important warnings always textual (not color-only)

---

11. Loveable AI Prompt (You Can Copy/Paste)

If you want to generate UI quickly:

> “Design a dark-mode crypto launchpad UI inspired by pump.fun and Solana trading dashboards.
> Use a near-black background, flat rectangular cards, minimal borders, and a single orange accent color for primary actions.
> Typography should be clean and data-focused (Inter-style).
> Emphasize charts, progress bars, and urgency indicators like ‘Next price jump’.
> Avoid playful fonts, heavy shadows, or NFT-gallery aesthetics.
> The vibe should feel like a serious casino for speculative traders.”

Final Check

This style:

- ✅ feels meme-native
- ✅ avoids unserious branding
- ✅ matches Mammoth’s mechanics
- ✅ scales from degen to builder
- ✅ is fast to buil

GLOBAL FIGMA AI SETUP PROMPT (RUN FIRST)

Prompt

> Create a dark-mode crypto web app design system inspired by Solana meme launchpads.
> Use a near-black background, flat rectangular cards, thin borders, and a single orange accent color.
> Typography should be clean, data-first, and neutral (Inter-style).
> Avoid rounded toy UI, gradients, heavy shadows, playful fonts, or NFT gallery aesthetics.
> Design should feel like a serious casino interface for speculative traders.

Run this once so Figma establishes a baseline.

PAGE 1 — Landing / Discovery

Prompt

> Design a dark-mode crypto launchpad landing page inspired by pump.fun.
> The page should list active token launches as flat rectangular cards.
> Each card must display: token name and ticker, current price in SOL, progress bar showing percent sold, a small sparkline chart, and a “View” button.
> Include sorting tabs at the top: Newest, Fastest Growing, Most Raised, Ending Soon.
> No wallet connection required for viewing.
> Use a single orange accent color for progress bars and buttons.

PAGE 2 — Project Page (Open Cycle)

Prompt

Design a project detail page for an active token launch.
The layout should be two-column: a large price chart on the left and a “Current Cycle” panel on the right.
The cycle panel must include: curve type (Step), allocation remaining, progress bar, current price in SOL, “Next price jump in X tokens” indicator, user rights remaining, and a prominent “Buy Tokens” button.
Use a dark background, thin borders, and an orange accent color.
The design should emphasize urgency without clutter.

PAGE 3 — Buy Modal

Prompt

Design a modal for purchasing tokens during a live launch cycle.
The modal should include: token amount input, total SOL cost, platform fee breakdown, price impact, rights usage indicator, slippage selector, and a confirm purchase button.
Use a dark theme with flat panels, aligned numeric values, and a clear primary action button in orange.
The modal should feel fast, serious, and frictionless.

PAGE 4 — Project Page (Closed Cycle / Trading)

Prompt

Design a project page for a completed token launch.
Display a large price chart at the top and Buy / Sell buttons below it.
Include a subtle notice that trades are routed via an aggregator and that a 2% fee applies only when trading through the platform.
Maintain the same dark, flat, data-focused aesthetic as the open cycle page.

PAGE 5 — About / Tokenomics Tab

Prompt

> Design an “About / Tokenomics” tab for a crypto launch project.
> The layout should present structured information including project description, supply mode, total supply, rights policy, and treasury routing.
> Use simple typography, muted colors, and clear section headers.
> This page should feel informational and transparent, not promotional.

PAGE 6 — Creator Dashboard

Prompt

> Design a creator dashboard for managing a token launch.
> The dashboard should display token supply statistics, active cycle status, past cycle history in a table, and treasury balances.
> Include buttons for opening and ending cycles.
> Use a dark theme with flat panels, thin borders, and minimal visual noise.

PAGE 7 — Launch Project Wizard

Prompt

> Design a multi-step wizard for launching a new token.
> Steps should include: token details, supply mode selection, allocation setup, and review before deployment.
> Use clear step progression, simple form inputs, and warning text for advanced options.
> Keep the design minimal, fast, and data-driven.

PAGE 8 — Create Cycle Flow

Prompt

> Design a flow for creating a new minting cycle for a token.
> The flow should include sections for token allocation, bonding curve selection with

live price preview, rights configuration, and treasury routing.
Default the curve to a step-based curve and highlight it as recommended.
Use a dark, flat design with orange accent highlights.

IMPORTANT FIGMA AI TIPS (READ THIS)

- Run prompts **one page at a time**
- After each page:
    - Ask Figma AI:
      “Refine spacing and align numeric values for readability”
- If it gets too playful, correct it with:
  “Reduce visual noise and remove any playful or decorative elements”

WHAT YOU’LL HAVE AFTER THIS

If you run these prompts, you will end up with:

- A full Mammoth MVP design
- Consistent visual language
- Screens ready for dev handoff
- Zero ambiguity about layout or intent

This is exactly the right moment to let AI do the heavy lifting.

Next steps (once designs exist)

1. Map **each screen → contract calls**
2. Generate **frontend component list**
3. Prepare **investor / builder demo**
4. Start **contract + frontend parallel build**

Ui flow Spec

Mammoth UI Flow Specification (MVP)

---

0. Global UI Principles

- **Wallet-first interaction**
  - Wallet connection required only for actions
  - Browsing requires no login
- **Pump.fun–familiar layout**
  - Minimalist
  - Chart-first
  - Clear CTAs
- **Protocol transparency**
  - Tokenomics, cycle rules, and fees always visible
- **No governance UI in MVP**
- **SOL-only flows**

---

1. Landing Page / Discovery

Purpose

Surface active opportunities and drive speculative engagement.

Visible to: Everyone (no wallet required)

Components

- **Header**
  - Mammoth logo
  - “Launch Project” button
  - “Connect Wallet” (top-right)
- **Offering List**
  - Sort options:
    - Newest
    - Most SOL Raised (24h / all-time)
    - Fastest Growing
    - Ending Soon
  - Each row/card shows:

        - Token name + ticker
        - Current cycle status (OPEN / CLOSED)
        - Price (current curve price)
        - % of cycle allocation sold
        - Mini sparkline
        - “View” button
- **No algorithmic curation**
    - Ordering is transparent
    - No endorsements

---

2. Project Page (Investor-Facing)

Purpose

Enable informed participation + fast buying.

Visible to: Everyone

Actions require: Wallet connected

2.1 Top Section (Above the Fold)

- **Token name + ticker**
- **Creator wallet (shortened, clickable)**
- **Cycle status badge**
    - OPEN / COMPLETED / TERMINATED / UPCOMING
- **Primary chart**
    - TradingView-style
    - Shows:
        - price
        - volume
        - cycle boundaries

2.2 Cycle Panel (Critical)

Displays current or most recent cycle.

If cycle is OPEN:

- Curve type (Step / Linear / Exp-Lite)
- Allocation:
    - total
    - remaining
- Current price
- Next price jump indicator (Step curve):
    - “Next step in X tokens”
- Rights status (if enabled):
    - “Your remaining rights: X tokens”
- Countdown:
    - optional (if creator set soft timing expectations)

Primary CTA

- Buy button

2.3 Buy Modal (OPEN Cycle)

Inputs

- Amount to buy (tokens)
- Slippage tolerance (advanced toggle)

Displays

- SOL required
- Platform fee (2%)
- Price impact
- Rights usage (if applicable)

Actions

- Buy & Confirm (wallet popup)

Errors handled clearly

- Exceeds rights
- Slippage exceeded
- Cycle closed mid-tx

2.4 Secondary Trading (Post-Cycle)

If cycle CLOSED:

- “Buy / Sell” buttons
- Routed via aggregator
- Fee disclosure:
    - “2% fee applies only when trading via Mammoth”

No custody.
No token taxes.

2.5 About / Tokenomics Tab

Sections

- Project description (freeform markdown)
- Supply mode:
    - Fixed or Elastic
- Hard cap status (if set)
- Cycle history table:
    - cycle ID
    - allocation
    - price range
    - completion status
- Rights policy summary
- Treasury routing breakdown
- Protocol fees disclosure

---

3. Creator Flow

Entry Point

“Launch Project” button (wallet required)

3.1 Token Creation Wizard

Step 1 – Basics

- Token name

- Ticker
- Description
- Creator treasury address (default = wallet)

Step 2 – Supply Mode

- Fixed Supply (default)
- Elastic Supply (advanced)
    - Warning modal:
        - “Elastic supply requires rights-based issuance”
        - “Hard cap can be set later, irreversibly”

Step 3 – Initial Allocation

- Public allocation (for cycles)
- Treasury allocation
- Live validation (fixed vs elastic rules)

Step 4 – Review & Create

- Summary screen
- Deploy token (wallet confirmation)

3.2 Cycle Creation Flow

Step 1 – Allocation

- Tokens allocated to this cycle
- Validation vs remaining supply / cap

Step 2 – Curve Selection

- Step (default, recommended)
- Linear
- Exp-Lite

Live price preview chart required.

Step 3 – Rights Toggle

- Enabled (mandatory for elastic)
- Disabled (fixed only)
- Explanation tooltip

Step 4 – Treasury Routing

- % to creator treasury
- % to reserve
- % to sink/burn
- Protocol fee shown but immutable

Step 5 – Create Cycle (DRAFT)

3.3 Open Cycle

From dashboard:

- “Open Cycle” button
- Warning:
    - “Parameters will be locked”
    - “Rights snapshot will be taken”

After confirmation:

- Cycle becomes OPEN
- Snapshot executed
- Cycle visible on discovery page

3.4 End Cycle Early

If OPEN:

- “End Cycle” button
- Confirmation modal:
    - “Unminted tokens will be unavailable unless reallocated”
- Cycle becomes TERMINATED

3.5 Reallocate Unissued Tokens

From terminated cycle:

- Select amount

- Select target DRAFT cycle
- Validation against constraints
- Confirm transaction

---

4. Creator Dashboard

Sections

- Token overview:
    - total supply
    - minted
    - remaining
- Active cycle status
- Past cycles table
- Treasury balances (SOL)
- Protocol fees paid (informational)

No governance controls.

---

5. Wallet & Permissions

- Wallet connection required for:
    - buying
    - selling (via Mammoth)
    - creating tokens
    - opening/ending cycles
- No email, no accounts, no KYC

---

6. MVP Explicit Exclusions (UI)

The UI does NOT include:

- Governance voting
- NFT passes
- Comments or forums
- Social feeds
- Recommendations

- Auto-marketing tools

These are future layers.

---

7. UX Success Criteria (MVP)

The UI is successful if:

- A user can buy in <30 seconds
- A creator can launch a cycle in <5 minutes
- Rights usage is clearly visible
- Step curve “next jump” urgency is obvious
- No user confusion about fees or supply

---

8. Hand-off Notes for Implementation

- This UI can be built with:
  - Next.js
  - Solana Wallet Adapter
  - Jupiter SDK
  - Charting library (TradingView / Lightweight Charts)
- Loveable AI can generate:
  - layout
  - components
  - state wiring
- Contract integration is straightforward once PDAs are defined

---

Contract Architecture

Good — this is the *right* next layer.
Now we move from **theory → enforceable reality**.

I'm going to do three things:

1. Define the **minimum contract set** Mammoth needs
2. Lay out **contract variants** (what can be modular vs fixed)
3. Call out **design decisions you must lock in vs defer**

No fluff, no Solana-specific code yet — just architecture.

---

1. The irreducible contract set (MVP)

At minimum, Mammoth needs **four conceptual contracts** (they can be combined or split later, but these roles must exist).

1. Token Contract

Purpose: represents ownership, nothing more.

Responsibilities:

- SPL token (or equivalent)
- Fixed or elastic supply flag set at creation
- Optional hard-cap state
- Mint authority controlled *only* by Cycle Manager
- No transfer taxes
- No hooks
- No governance

Key invariant:

> The token itself never decides *when* it mints — it only allows minting by an authorized cycle controller.

This keeps tokens clean and composable.

---

2. Cycle Manager Contract (core of Mammoth)

This is the heart of the protocol.

Responsibilities:

- Define cycles
- Enforce issuance rules
- Mint tokens during cycles
- Enforce bounded bonding curves
- Snapshot holders for rights
- Enforce elastic vs fixed rules
- Enforce hard-cap transition
- Route proceeds

This contract is where Mammoth *is*.

Key invariant:

> All issuance must pass through the Cycle Manager.

If someone can mint without it, Mammoth breaks.

---

3. Rights Controller (logical module, maybe separate)

Purpose: prevent forced dilution.

Responsibilities:

- Take end-of-cycle snapshots
- Calculate pro-rata rights
- Track exercised vs unused rights
- Enforce purchase caps per address
- Expire rights when cycle ends

This can be:

- a sub-module of Cycle Manager (simpler MVP), or
- a separate contract (cleaner, but more complex)

Key invariant:

> Rights must be enforceable even if the creator wants to bypass them.

This is where trust is won or lost.

---

4. Treasury Router

Purpose: remove discretion from fund flows.

Responsibilities:

- Receive proceeds
- Route X/Y/Z deterministically
- Send protocol fee to Mammoth treasury
- Send remainder to creator treasury
- Optional burn sink

Key invariant:

> Funds never pass through creator-controlled logic before routing.

This is how you avoid “trust me bro”.

---

2. Contract variants you should explicitly support

Now let’s talk **variants**, because Mammoth is not one-size-fits-all.

Variant A: Fixed Supply Token (default)

Enabled

- Predefined total supply
- Predefined public allocation
- Cycles draw down from fixed pool
- No minting after pool exhausted

Disabled

- Elastic minting
- Supply expansion
- Rights after final cycle

Simplicity level

🟢 Lowest
This will be the majority of launches.

Variant B: Elastic Supply Token (uncapped)

Enabled

- Minting via cycles only
- Rights mandatory
- Supply grows over time
- No emissions

Disabled

- Free minting
- Off-cycle issuance

Risk profile

🟡 Medium
Market prices future cycles as risk.

Variant C: Elastic → Hard-Capped (irreversible)

This is not a separate mode — it's a **state transition**.

Additional contract logic

- `setHardCap(maxSupply)`
- Valid only if:
  - called once
  - maxSupply ≥ current minted
- After call:
  - minting permanently disabled beyond cap
  - rights permanently disabled
  - Cycle Manager switches to fixed logic

Key invariant

> This function must be one-way and uncallable again.

This is your strongest credibility lever.

---

3. Cycle-level variants (important)

Each **cycle** can vary independently, even under the same token.

Things a creator can configure per cycle:

- allocation size
- bonding curve shape (linear, exponential, stepped)
- start price
- treasury routing percentages (within bounds)
- whether rights apply (forced in elastic mode, optional in fixed mode)

Things they **cannot** change:

- supply mode
- rights enforcement rules
- hard-cap status
- protocol fee
- immutability once cycle opens

---

4. The “End Cycle” function (we need to be precise)

You already identified this as necessary.

Required function

endCycle(cycleId)

Rules:

- callable only by creator
- callable only while cycle is open
- immediately disables minting
- locks pricing
- freezes unminted allocation

Critically:

- unminted tokens do NOT return to treasury
- they do NOT disappear

- they are marked “unissued”

Reallocation requires:

- a new cycle
- same issuance constraints
- same rights logic

This prevents stealth repricing.

---

5. What must be immutable vs configurable

This is where most protocols screw up.

Immutable (locked forever)

- protocol fee (2%)
- protocol stake rate (2%)
- rights enforcement logic
- hard-cap irreversibility
- mint authority rules

Configurable at token creation

- supply mode (fixed / elastic)
- initial supply
- public vs treasury allocation

Configurable per cycle (before open)

- allocation size
- curve parameters
- treasury routing (within bounds)

Immutable once cycle opens

- everything about that cycle

---

6. What NOT to put on-chain (yet)

Resist temptation.

Do NOT encode:

- marketing claims
- roadmap promises
- timelines
- creator identity
- governance logic
- studio structures

Those live in UI and off-chain metadata.

---

7. The biggest architectural risk (be aware)

The hardest part is **rights snapshots at scale**.

On Solana, this means:

- large holder sets
- snapshot performance
- edge cases with fast transfers
- snapshot at start of next cycle

This is solvable — but it’s the first real engineering challenge.

---

8. Sanity check: does this match Mammoth’s philosophy?

Yes.

- Clean token
- Strong issuance controller
- Explicit trust boundaries
- Minimal surface area
- Permissionless but disciplined

You’re designing **a capital formation engine**, not a DAO factory.

Where to go next (your choice)

From here, the logical next steps are:

1. **Exact state machine for Cycle Manager**
2. **Rights math + snapshot timing**
3. **Bonding curve primitives**
4. **Solana-specific constraints & accounts**
5. **Attack scenarios (how people try to cheat this)**

Tell me which one you want to tackle next, and we'll go deep.

Customer Profile

Customer Profile

1. Who Mammoth Is NOT For (Important to Say Out Loud)

Mammoth is **not** for:

❌ One-off meme creators

- People who want:
  - a single launch
  - a fast pump
  - no responsibility after
- Pump.fun already serves them better

❌ Traditional DAO builders

- People who want:
  - governance-first systems
  - long constitutions
  - slow consensus
- They want Juicebox, Aragon, or custom DAO stacks

❌ VC-backed startups

- They already have:
  - capital
  - structured fundraising
  - legal wrappers
- Mammoth's asymmetry is a liability to them

❌ "Passive yield" seekers

- No emissions
- No staking APY
- No guarantees

If someone asks:

> "How safe is this?"

They are **not** Mammoth's user.

---

2. Who Mammoth Is EXACTLY For (v1 Ideal Creator Profile)

Mammoth is for Repeat Builders with Asymmetric Narratives

More precisely:

> **Founders who expect to raise capital more than once, want meme-level upside early, and are willing to be publicly accountable over time — without surrendering control.**

Let's break that down.

The Mammoth v1 Creator

Psychographics

- Comfortable with volatility
- Comfortable being wrong in public
- Thinks in *cycles*, not launches
- Does not want governance telling them what to do
- Understands markets will punish them if they mess up

Behaviorally

- Already has:
  - a community
  - an audience
  - or a strong narrative
- Expects:
  - to build in stages
  - to ship between raises
  - to raise again if things go well

Crucially

They believe:

> "If I do good work, the market should let me raise again — but not for free."

That belief is **rare** — and it's Mammoth's edge.

Concrete Examples (Not Hypotheticals)

Mammoth is for people like:

- A studio founder launching:
    - Project A now
    - Project B in 6 months
    - Project C next year
      → wants continuity, not reinvention
- A creator with:
    - a strong meme
    - a real roadmap
    - no desire for DAO governance
      → wants to *earn* dilution, not hide from it
- A builder who says:
  "I don't want emissions, I want conviction."

That's your user.

---

3. Who Mammoth Is For on the Investor Side

This matters just as much.

Mammoth Investors Are:

- Speculators who:
    - understand risk
    - want asymmetry
    - don't expect safety rails
- People who:
    - *want* dilution to be optional
    - *want* future opportunities
    - don't want surprise inflation

They are **not**:

- DAO voters
- LP farmers
- Passive yield chasers

They are closer to:

- early public equity investors

- crypto-native venture tourists
- high-conviction degenerates who want rules, not promises

---

4. The Killer Insight (This Is the Core)

Mammoth is not a launchpad.

Mammoth is a reputation machine.

It allows the market to answer one question over time:

> "Has this builder earned the right to raise again?"

That is the **primitive** you're creating.

Everything else — cycles, rights, curves — serves that.

---

5. The One-Sentence Positioning (Internal Truth)

If you remember nothing else, remember this:

> **Mammoth is for builders who want meme-level asymmetry without one-shot extraction — and for investors who want chaos with rules.**

That sentence should guide:

- feature cuts
- UI decisions
- marketing tone
- who you say "no" to

---

6. Hard Truth (You'll Appreciate This)

Mammoth will:

- feel *too risky* for serious institutions
- feel *too disciplined* for pure degens

That’s not a weakness.
That’s the **moat**.

You’re building for the overlap that *doesn’t have a home yet*.

Economic Stress Test

Mammoth Economic Stress Test

We'll test Mammoth against **real market behavior**, not ideal users.

Baseline Assumptions (So We're Honest)

- Token has real secondary liquidity
- Participants are speculative, not loyal
- Most users:
    - do not read whitepapers
    - do not exercise rights unless incentivized
- Creators are imperfect and sometimes desperate
- Markets price **expected future supply**, not promises

Scenario 1: Cycle 1 Underperforms

Situation

- Cycle 1 allocation: 400k tokens
- Only 120k sell
- Price never reaches upper bands
- Creator terminates cycle

What Happens

✅ No forced failure

- Cycle does not "fail"
- No minimum raise
- No refunds
- No protocol embarrassment

✅ Supply integrity preserved

- Unsold tokens remain unavailable
- Allocation is not automatically reintroduced

⚠️ **Narrative damage**

- Market learns demand was weaker than expected
- Price may drift down post-cycle

Key Insight

This is *no worse* than a bad meme launch — and significantly better than:

- a rug
- inflation
- emergency governance changes

Verdict:
✔ Survives cleanly
⚠ Requires creator narrative competence

Scenario 2: Cycle 2 Cannibalizes Cycle 1 Demand

Situation

- Cycle 1 succeeds
- Market anticipates Cycle 2
- Buyers delay purchases
- Secondary price softens pre-cycle

This Is the Core Fear

You've articulated this correctly:

> "The market prices in future issuance regardless of timing."

What Mammoth Does Differently

1. Rights convert fear into optionality

Instead of:

> "I will be diluted"

Holders think:

> "I have a protected allocation"

This **reduces panic selling**, but does not eliminate it.

2. Bounded cycles cap damage

- No infinite minting
- No emissions drip
- No surprise inflation

3. Allocation sizing matters more than timing

Small future cycles = low inflation anxiety
Large future cycles = market punishment

Mammoth doesn't solve bad planning — it **exposes it early**.

Verdict

✔ Survivable
⚠ Strongly dependent on cycle sizing discipline
⚠ Market still prices dilution, just less violently

Scenario 3: Rights Are Largely Unused

Situation

- Snapshot taken
- Most holders ignore rights
- New buyers dominate cycle
- Early holders diluted voluntarily

Is This a Problem?

No. This is expected behavior.

This mirrors:

- equity rights offerings
- secondary offerings in public markets

The key is that:

- dilution was **opt-in**
- blame cannot be externalized

Hidden Benefit

Inactive holders get flushed over time
Active conviction consolidates ownership

This is **healthy**, not harmful.

Verdict

✔ Works as designed
✔ Filters passive capital

Scenario 4: Price Collapses Between Cycles

Situation

- Cycle ends
- Liquidity dries up
- Price trends down 60–80%
- Creator plans next cycle

What Happens Next?

Three paths:

A. Creator delays cycle

- Best case
- Market stabilizes
- Credibility preserved

B. Creator launches anyway

- Market punishes allocation pricing
- Cycle raises less capital
- Creator learns fast

C. Creator terminates platform usage

- Mammoth loses nothing
- No protocol risk

Importantly

There is **no reflexive death spiral** like:

- emissions-based DAOs
- yield tokens
- ponzi staking

Verdict

✔ Failure is localized
✔ Protocol is insulated

Scenario 5: Elastic Supply Causes Trust Decay

Situation

- Creator launches elastic
- Runs 2–3 cycles
- Market fears infinite dilution
- Price stagnates

This Is Real Risk

You mitigated it correctly by:

- forbidding continuous minting
- requiring rights
- allowing irreversible hard cap commitment

Reality Check

Elastic mode will:

- be used by few
- be punished faster
- self-select out bad actors

Which is **exactly what you want**.

Verdict

✔ Acceptable risk
✔ Correctly optional
⚠ Needs strong UI warnings

Scenario 6: Secondary Dump Before Guaranteed Curve Jump

This is the *nastiest edge case* you identified.

Situation

- Cycle 2 price floor > current market price
- Rational traders buy on secondary
- Holders frontrun curve jump
- Dump immediately after

Can Mammoth Prevent This?

No.
And **neither can traditional markets**.

This mirrors:

- IPO pops
- secondary arbitrage
- dilution cliffs

What Mammoth Does Instead

- Makes the behavior legible
- Forces creators to price cycles realistically
- Encourages:
    - smaller allocations
    - longer gaps
    - softer curves

Verdict

⚠ Real arbitrage risk
✔ Acceptable
✔ Exposes bad design quickly

Overall Stress Test Verdict

Mammoth does not:

- guarantee price stability
- prevent speculation
- save bad projects
- eliminate dilution anxiety

Mammoth does:

- prevent structural death spirals
- convert dilution into choice
- localize failure
- preserve upside asymmetry
- allow repeat fundraising without emissions

Hard Truth (Important)

If Mammoth fails, it will fail for **human reasons**, not protocol reasons:

- bad timing
- bad sizing
- bad narratives
- unrealistic expectations

That's exactly where crypto *should* put risk.

Final Assessment

Mammoth is economically sound.
It does not promise safety — it promises **fairness under chaos**.

That's the correct promise for crypto.

Cycle Manager

Mammoth Protocol

Cycle Manager Specification

---

1. Purpose

The Cycle Manager is the core issuance engine of the Mammoth protocol.
It enforces discrete, event-based token issuance through bounded minting cycles, while supporting rights-based anti-dilution, flexible supply configurations, and deterministic treasury routing.

All token minting under Mammoth MUST occur through the Cycle Manager.
No continuous emissions, inflation, or external mint authorities are permitted.

---

2. High-Level Guarantees (Invariants)

The Cycle Manager enforces the following invariants:

1. **Single Mint Authority**
   The Cycle Manager PDA is the sole mint authority for the token.
2. **Discrete Issuance Only**
   Tokens may only be minted during an OPEN cycle.
3. **Cycle Immutability**
   Once a cycle is opened, its parameters cannot be changed.
4. **Bounded Minting**
   Each cycle has a fixed allocation that cannot be exceeded.
5. **Rights-Based Protection**
   When enabled, dilution protection is enforced via non-transferable, cycle-specific rights.
6. **Supply Discipline**
   - Fixed supply tokens may never exceed their initial cap.
   - Elastic supply tokens may mint only through cycles.
   - A hard cap, once set, is irreversible.
7. **Market Neutrality**
   The protocol does not gatekeep, curate, endorse, or underwrite projects.
8. **Fee Neutrality**
   Fees are charged only on Mammoth-routed transactions. External trades are untouched.

---

3. Core Objects

3.1 TokenConfig (PDA, One per Token)

Stores global configuration and supply constraints.

Fields

- `mint` – SPL token mint
- `creator` – creator authority
- `supply_mode` – FIXED | ELASTIC
- `hard_cap` – optional u64 (None until set)
- `minted_total` – total minted supply
- `public_remaining` – remaining public allocation (FIXED only)
- `protocol_fee_bps` – fixed at 200 (2%)
- `protocol_stake_bps` – fixed at 200 (2%)
- `creator_treasury` – SOL destination
- `protocol_treasury` – Mammoth treasury
- `payment_asset` – SOL (MVP)
- `secondary_routing` – enabled (UI + aggregator)
- `status` – ACTIVE | CAPPED

Invariants

- Mint authority is permanently assigned to Cycle Manager PDA.
- No minting outside `buy_in_cycle`.

3.2 Cycle (PDA, One per Cycle)

Defines a discrete minting event.

Fields

- `cycle_id`
- `state` – DRAFT | OPEN | CLOSED_COMPLETED | CLOSED_TERMINATED
- `allocation` – token supply for the cycle
- `sold` – tokens minted so far
- `curve_type` – STEP | LINEAR | EXP_LITE

- `curve_params` – curve-specific parameters
- `snapshot_slot` – slot used for rights snapshot
- `merkle_root` – root of snapshot balances
- `snapshot_total_supply` – total supply at snapshot
- `opened_at_slot`
- `closed_at_slot`
- `routing_override` – optional treasury routing

Invariants

- Cycle parameters are immutable after OPEN.
- Minting is allowed only while OPEN.

3.3 UserRights (PDA, One per User per Cycle)

Tracks rights entitlement and usage.

Fields

- `user`
- `cycle_id`
- `entitlement_max`
- `used`
- `initialized`

Properties

- Rights are:
  - non-transferable
  - cycle-specific
  - enforced only while cycle is OPEN
  - expired once cycle closes

---

4. Supply Modes

4.1 Fixed Supply

- Total supply defined at genesis.
- All minting occurs through cycles.

- No inflation or emissions.
- Protocol stake (2%) is minted at genesis.

4.2 Elastic Supply

- No maximum supply initially.
- Minting allowed only through cycles.
- Rights-based issuance is mandatory.
- Protocol stake (2%) is minted proportionally per cycle.

4.3 Hard-Cap Commitment

- Creator may set an irreversible hard cap at any time.
- Once set:
    - total supply becomes fixed
    - future cycles must fit under cap
    - rights MAY continue while cycles exist

This transition is on-chain and irreversible.

---

5. Rights-Based Issuance

5.1 Snapshot Rule (Canonical)

Rights are determined by **token balances at the moment a cycle opens**.

- Snapshot occurs at `open_cycle`
- Snapshot data is provided as a Merkle root
- No historical comparisons
- No holding-period logic

5.2 Entitlement Formula

For a given cycle:

```
entitlement =
floor(
  cycle_allocation × user_balance_at_snapshot
  ÷ snapshot_total_supply
)
```

- Users may purchase up to their entitlement.
- Unused rights expire at cycle close.

5.3 Applicability

- Mandatory for elastic supply cycles.
- Optional for fixed supply cycles.
- Rights persist after hard cap as long as cycles exist.

---

6. Bonding Curves

Each cycle uses one bounded pricing curve.

6.1 Step Curve (Default)

- Price increases in discrete steps.
- Parameters:
    - start_price
    - step_size
    - step_increment
- Meme-native, milestone-driven, highly gamified.

6.2 Linear Curve

- Price increases smoothly from start_price to end_price.
- Predictable and neutral.

6.3 Exp-Lite Curve

- Early price low, accelerates late.
- Bounded exponential growth.
- Strong early asymmetry without runaway pricing.

Curve type and parameters are immutable after cycle open.

---

7. Cycle Lifecycle

7.1 Create Cycle

`create_cycle(...)`

- Creates cycle in DRAFT state.
- Validates allocation against supply constraints.

7.2 Open Cycle

`open_cycle(cycle_id, merkle_root, snapshot_total_supply)`

- Transitions DRAFT → OPEN.
- Records snapshot_slot and rights root.
- Freezes parameters.

7.3 Buy in Cycle

`buy_in_cycle(cycle_id, amount_out, max_amount_in, proof, balance_at_snapshot)`

Flow:

1. Validate cycle OPEN
2. Verify Merkle proof (if rights enabled)
3. Initialize UserRights if needed
4. Enforce entitlement limit
5. Compute price from curve
6. Enforce slippage guard
7. Collect SOL
8. Take 2% protocol fee
9. Route proceeds
10. Mint tokens
11. Mint protocol stake (elastic only)
12. Update state
13. Auto-close if allocation exhausted

7.4 End Cycle (Early Termination)

`end_cycle(cycle_id)`

- Creator-only
- Transitions OPEN → CLOSED_TERMINATED
- Freezes sold amount
- Unminted tokens become unavailable unless reallocated

7.5 Reallocate Unissued Tokens

```
reallocate_unissued(from_cycle, to_cycle, amount)
```

Rules:

- Source cycle must be CLOSED_TERMINATED
- Destination must be DRAFT
- Cannot violate:
    - supply mode
    - hard cap
    - rights requirements
- Cannot weaken issuance constraints

---

8. Treasury Routing

On every purchase:

- SOL is routed deterministically:
    - X% → creator treasury
    - Y% → reserve
    - Z% → sink/burn (optional)
- Protocol fee (2%) is routed to Mammoth treasury.
- No funds remain idle in the contract.

---

9. Secondary Market Routing

- Mammoth provides Buy/Sell UI post-cycle.
- Trades are routed through aggregators (e.g. Jupiter).
- 2% fee applies only when routed through Mammoth.
- No token-level taxes or restrictions.
- External trades are unaffected.

---

10. Failure Semantics

- Cycles are not guaranteed to complete.
- If allocation is not exhausted:
    - cycle may remain open
    - or be terminated by creator
- Unminted tokens remain unavailable unless explicitly reallocated.
- The protocol does not guarantee success, pricing, or liquidity.

---

11. Non-Goals (Explicit)

The Cycle Manager does NOT:

- enforce governance
- curate projects
- impose lockups
- guarantee returns
- restrict secondary markets
- evaluate legitimacy

Mammoth is an issuance framework, not a guarantor.

---

12. Summary

The Cycle Manager enables:

- repeatable capital formation
- meme-native gamification
- disciplined issuance
- voluntary dilution
- supply credibility enforced by code

This document defines the complete MVP behavior of Mammoth’s core issuance engine.

Tab 10

Mammoth v1 — Contract Architecture & Build Plan (Solana)

Design Goals (Re-anchoring)

- Deterministic, legible issuance
- No governance required in MVP
- Creator autonomy, protocol neutrality
- Rights-based issuance without NFTs (v1)
- Solana-native, SPL-only
- Minimal surface area, composable later

High-Level Architecture

Mammoth v1 consists of **four on-chain programs** (or one program with four logical modules if preferred):

1. **Token Factory**
2. **Cycle Manager**
3. **Rights Registry**
4. **Treasury Router**

UI, charts, and discovery are **off-chain** and read-only.

---

1. Token Factory Program

Purpose

Creates Mammoth-compatible tokens with enforced issuance rules.

Responsibilities

- Mint SPL token
- Lock supply mode (fixed or elastic)
- Initialize allocation buckets

- Register token with Mammoth registry

Key State (PDA)

TokenConfig

token_mint: Pubkey
creator: Pubkey
supply_mode: Fixed | Elastic
hard_cap: Option<u64>
total_supply: u64
public_allocation_remaining: u64
treasury_allocation: u64
hard_cap_committed: bool
created_at: i64

Invariants

- Tokens **cannot be minted** outside Cycle Manager
- Elastic mode can mint *only through cycles*
- Hard cap can be set **once**, irrevocably

Instructions

- `create_token()`
- `commit_hard_cap(cap: u64)`
- `register_allocation(public, treasury)`

---

2. Cycle Manager Program (Core)

Purpose

Controls all issuance events (“cycles”).

This is the heart of Mammoth.

Cycle Lifecycle

Draft → Open → Closed (Completed or Terminated)

No paused state. No partial guarantees.

Key State (PDAs)

CycleConfig
cycle_id: u64
token_mint: Pubkey
allocation: u64
curve_type: Step | Linear | Exponential
curve_params: bytes
start_time: i64
end_time: Option<i64>
rights_required: bool
status: Draft | Open | Closed

CycleAccounting
tokens_sold: u64
sol_collected: u64

Supported Curve Types (v1)

Default: Step Curve

- Discrete price bands
- Meme-native
- Simple mental model

Other curves are optional flags:

- Linear
- Exponential

All curves are **bounded by allocation**.

Instructions

- `create_cycle(config)`
- `open_cycle()`
    - Snapshot rights (see Rights Registry)
- `buy_tokens(amount)`
- `terminate_cycle()` *(creator-only)*
- `close_cycle()` *(auto when allocation exhausted)*

Important Rules

- If a cycle underperforms, it simply remains open **until terminated or sold out**
- Terminated cycles:
    - Return unsold allocation to future pool
    - Do NOT mint remaining tokens
- No minimum raise
- No refunds

---

3. Rights Registry Program

Purpose

Implements **rights-based anti-dilution** without NFTs (v1).

Rights Model (Chosen Variant)

✅ **Snapshot-on-open**
✅ Rights are **non-transferable**
✅ Rights persist across cycles until exercised or expired

Key State

RightsSnapshot

cycle_id: u64
snapshot_slot: u64

UserRights

user: Pubkey
token_mint: Pubkey
cycle_id: u64
max_allocation: u64
exercised: u64

Flow

1. When a cycle opens:
    - Snapshot balances of token holders
2. Each holder receives:
    - Pro-rata purchase cap for the cycle
3. When buying:
    - Rights are checked before allowing mint

Notes

- Rights do **not** require end-of-cycle snapshots
- Rights persist as long as cycles continue
- Rights cease permanently when:
    - Hard cap is committed
    - No future cycles exist

---

4. Treasury Router Program

Purpose

Deterministically routes proceeds.

Routing Logic (v1 fixed)

On every purchase:

- X% → creator treasury
- Y% → reserve vault (SOL / wrapped asset)
- Z% → optional sink (burn / protocol)

All percentages are **immutable per token**.

Key State

```
TreasuryConfig {
  creator_treasury: Pubkey
  reserve_vault: Pubkey
  protocol_fee_vault: Pubkey
  routing_percentages: (u8, u8, u8)
}
```

Protocol Fees

- 2% of buys/sells routed **only if executed through Mammoth**
- No enforcement on external markets
- Protocol also receives:
    - 2% of total supply (fixed mode)
    - OR 2% of all minted tokens (elastic mode)

Program Interaction Diagram (Mental Model)

```
User Wallet
  ↓
Mammoth UI
  ↓
Cycle Manager ——→ Rights Registry
  ↓
Treasury Router
  ↓
SPL Token Mint
```

Build Order (Critical)

Phase 1 — Foundations

1. Token Factory

2. Treasury Router
3. Registry wiring

Phase 2 — Issuance

4. Cycle Manager (Step curve only at first)
5. Rights Registry integration

Phase 3 — UX Completeness

6. Secondary trading routing
7. Cycle termination logic
8. Hard cap transition

What We Deliberately Did NOT Include (v1)

- Governance
- NFTs / passes
- Team vesting logic
- Anti-bot mechanics
- Reputation systems
- Discovery incentives

All of these are **layerable later**.

What This Architecture Buys You

- Meme-native asymmetry ✔
- Repeatable capital formation ✔
- No inflation anxiety ✔
- No protocol overreach ✔
- Clear failure modes ✔
- Composable future ✔

Most importantly:
this is buildable by 1–2 solid Solana devs.
